

GLOBALREMIT

Gobernanza y seguridad de datos

Manual técnico integrado · Etapa 4

Gobernanza de datos · Matriz RBAC · Autenticación MongoDB ·
Cifrado · Enmascaramiento · Auditoría

Documento consolidado

Julio de 2026

Contenido

Contenido.....	2
Etapa 4 - Seguridad Y Gobernanza De La Capa De Datos	4
1. Objetivo.....	4
2. Alcance De Seguridad.....	4
3. Clasificacion De Datos	5
4. Controles Propuestos.....	5
5. Criterio De Cierre	6
Etapa 4 - Gobernanza De Datos GlobalRemit	7
1. Objetivo.....	7
2. Dominios De Datos.....	7
3. Clasificacion Y Acceso	7
4. Politicas De Gobierno.....	8
5. Indicadores De Calidad	8
6. Ciclo De Vida	8
Etapa 4 - Matriz RBAC GlobalRemit	9
1. Objetivo.....	9
2. Roles PostgreSQL Propuestos	9
3. Matriz De Permisos PostgreSQL.....	10
4. Datos Sensibles Y Restricciones	11
5. Roles MongoDB Propuestos	11
6. Segregacion De Funciones	12
Etapa 4 - Autenticacion Y RBAC En MongoDB Replica Set.....	13
1. Objetivo.....	13
2. Concepto De Seguridad.....	13
3. Precauciones	13
4. Evidencia Inicial.....	14
5. Crear El Administrador Inicial	14
6. Generar La Llave Interna	14

7. Configurar Los Tres Nodos	15
8. Recrear Sin Borrar Datos	15
9. Demostrar El Bloqueo Anonimo	16
10. Acceso Administrativo	16
11. Crear Roles Funcionales	16
12. Probar Menor Privilegio	16
13. Failover Con Autenticacion	17
14. Problemas Frecuentes	18
15. Matriz De Evidencias	18
16. Criterio De Cierre	18
Etap4 - Cifrado En Transito Y En Reposo	19
1. Objetivo	19
2. Cifrado En Transito	19
2.1 PostgreSQL Patroni	19
2.2 MongoDB Replica Set	19
3. Cifrado En Reposo	19
3.1 Volumenes Y Sistema Operativo	19
3.2 Datos Sensibles A Nivel Aplicacion	20
4. Gestion De Secretos	20
5. Evidencia Esperada	20
Etap4 - Politica De Enmascaramiento Y Anonimizacion	21
1. Objetivo	21
2. Datos A Proteger	21
3. Reglas De Enmascaramiento	21
4. Vistas Seguras	21
5. Anonimizacion Para Desarrollo	22
6. Gobierno Del Dato	22
Etap4 - Configuracion De Auditoria	23
1. Objetivo	23
2. Eventos A Auditar	23

3. Auditoria PostgreSQL	23
4. Auditoria MongoDB	24
5. Retencion	24
6. Revision	24
Estado de avance de la etapa 4	25
Próximos pasos recomendados	25
Anexo A - Comandos utilizados y evidencias	27
Comandos utilizados	27
Autenticación MongoDB	29
Comandos utilizados	29

Etapa 4 - Seguridad Y Gobernanza De La Capa De Datos

1. Objetivo

Definir e implementar controles de seguridad y gobernanza para la arquitectura de datos de GlobalRemit, protegiendo informacion sensible de clientes, beneficiarios, remesas, cumplimiento, riesgo, liquidacion e integraciones.

La etapa cubre los cuatro entregables solicitados oficialmente:

Entregable oficial	Archivo principal
Matriz de Control de Acceso Basado en Roles (RBAC)	10_matriz_rbac.md
Autenticacion y RBAC ejecutable en MongoDB	10_autenticacion_mongodb.md, 10_mongodb_rbac.js
Scripts de cifrado de datos en reposo y transito	10_cifrado_tls_reposo.md y scripts asociados
Configuracion del sistema de auditoria	10_auditoria.md, 10_auditoria_postgresql.sql
Politica de enmascaramiento y anonimizacion	10_politica_enmascaramiento_anonimizacion.md, 10_enmascaramiento_anonimizacion.sql

2. Alcance De Seguridad

La seguridad se disena sobre la arquitectura final ya implementada:

```

Usuario / analista
-> Django API Gateway
-> HAProxy Patroni
-> PostgreSQL Leader
-> replicas PostgreSQL

Generador / pipeline
-> PostgreSQL
-> productos analiticos MongoDB replica set

```

La capa SQL conserva la verdad transaccional. MongoDB conserva productos analiticos derivados. Por ello, los privilegios se asignan segun el principio de menor privilegio y segun el tipo de dato: publico, interno, confidencial o sensible.

3. Clasificacion De Datos

Clasificacion	Ejemplos	Tratamiento
Publico	catalogos de paises, monedas y canales	Lectura amplia controlada.
Interno	estados operativos, reglas de corredor, metricas agregadas	Acceso por areas autorizadas.
Confidencial	remesas, movimientos, liquidaciones, cotizaciones	Acceso por rol funcional.
Sensible	documentos, cuentas, Travel Rule, AML, KYC, datos personales	Cifrado, auditoria y enmascaramiento.

4. Controles Propuestos

Control	Aplicacion en GlobalRemit
RBAC	Roles separados para aplicacion, operaciones, cumplimiento, analitica, auditoria y DBA.
Cifrado en transito	TLS entre clientes, API Gateway, HAProxy y PostgreSQL/MongoDB.
Cifrado en reposo	Volumenes cifrados, secretos protegidos y columnas sensibles cifradas a nivel aplicacion.
Auditoria	Registro de DDL, cambios DML sensibles, accesos privilegiados y eventos de seguridad.
Enmascaramiento	Vistas seguras para ambientes analiticos y no productivos.
Segregacion de funciones	Ningun rol operativo debe tener permisos de DBA completos.
Trazabilidad	Uso de logs, tablas de auditoria y Outbox para eventos de negocio.

5. Criterio De Cierre

La etapa se considera completa cuando:

1. Existen roles y permisos documentados para PostgreSQL y MongoDB.
2. MongoDB rechaza acceso anonimo y conserva el failover con autenticacion.
3. Existe un plan de cifrado en reposo y transito alineado con Docker/Patroni/MongoDB.
4. Existe una configuracion de auditoria para operaciones criticas.
5. Existen reglas de enmascaramiento para datos sensibles.
6. Los scripts pueden revisarse y ejecutarse de forma controlada en ambiente academico.

Etapa 4 - Gobernanza De Datos GlobalRemit

1. Objetivo

Definir responsabilidades, propietarios, clasificacion, calidad y ciclo de vida de los datos para que la arquitectura SQL y NoSQL de GlobalRemit sea gobernable, auditable y segura.

2. Dominios De Datos

Dominio	Fuente principal	Propietario	Consumidores
Cientes e identidad	PostgreSQL <code>customer</code>	Operaciones / Cumplimiento	Operaciones, cumplimiento, riesgo, analitica.
Remesas	PostgreSQL <code>remittance</code>	Operaciones	API, operaciones, finanzas, analitica.
FX y corredores	PostgreSQL <code>fx</code>	Tesoreria / Producto	API, operaciones, reporteria.
Cumplimiento	PostgreSQL <code>compliance</code>	Cumplimiento	Cumplimiento, auditoria, riesgo.
Liquidacion	PostgreSQL <code>settlement</code>	Finanzas	Finanzas, auditoria, operaciones.
Integracion	PostgreSQL <code>integration</code>	Tecnologia	API, pipeline, auditoria tecnica.
Analitica documental	MongoDB <code>globalremit_analytics</code>	Analitica / Riesgo	Dashboards, fraude, cumplimiento.

3. Clasificacion Y Acceso

Clasificacion	Reglas
Publico	Puede exponerse en documentacion y catalogos no sensibles.
Interno	Uso dentro de la organizacion, sin datos personales completos.
Confidencial	Requiere rol funcional y registro de acceso.
Sensible	Requiere menor privilegio, enmascaramiento, cifrado y auditoria.

4. Politicas De Gobierno

1. PostgreSQL es la fuente oficial para transacciones y cumplimiento.
2. MongoDB almacena productos derivados, no sustituye la verdad transaccional.
3. Todo dato sensible debe tener propietario y regla de acceso.
4. Los datos analiticos deben generarse desde fuentes trazables.
5. Los cambios DDL deben pasar por revision tecnica.
6. Las cuentas de servicio deben ser distintas de las cuentas humanas.
7. Los ambientes no productivos deben usar datos sinteticos o enmascarados.
8. Los eventos Outbox sirven como mecanismo de trazabilidad de integracion.

5. Indicadores De Calidad

Indicador	Regla
Compleitud	Remesa autorizada debe tener cliente, beneficiario, cotizacion, detalle financiero y estado.
Integridad	Las FK y validadores JSON deben rechazar referencias o documentos invalidos.
Trazabilidad	Cada cambio de estado debe tener historial.
Consistencia	El estado actual debe coincidir con el ultimo evento de historial.
Actualidad	Productos MongoDB deben registrar <code>updated_at</code> , <code>computed_at</code> o <code>source_watermark</code> .
Seguridad	Datos sensibles no deben exponerse sin rol autorizado.

6. Ciclo De Vida

Etapas	Control
Creacion	Validaciones SQL/JSON y permisos RBAC.
Uso operacional	Auditoria y restricciones de menor privilegio.
Uso analitico	Vistas enmascaradas y productos derivados.
Retencion	Conservacion segun regulacion financiera y auditoria.
Eliminacion	Evitar borrado fisico ordinario; preferir baja logica o retencion controlada.

Etapa 4 - Matriz RBAC GlobalRemit

1. Objetivo

Definir roles de acceso para PostgreSQL y MongoDB aplicando el principio de menor privilegio. Los permisos se separan por funcion de negocio, tipo de dato y responsabilidad operativa.

2. Roles PostgreSQL Propuestos

Rol	Proposito	Acceso principal
gr_app_writer	API Gateway y servicios transaccionales	Lectura de catalogos; escritura controlada en remesas, FX, cumplimiento operativo e integracion.
gr_ops_readonly	Operaciones y mesa de servicio	Lectura de remesas, estados, liquidaciones y catalogos; sin datos sensibles completos.
gr_compliance_officer	Cumplimiento, AML, KYC y Travel Rule	Lectura/escritura en compliance ; lectura de clientes y remesas necesarias.
gr_risk_analyst	Analisis de riesgo y fraude	Lectura de remesas, riesgo y vistas enmascaradas; sin escritura transaccional.
gr_finance_settlement	Liquidacion y conciliacion	Lectura de remesas y movimientos; escritura en settlement .
gr_data_analyst	Analitica y reporteria	Lectura de vistas enmascaradas y productos agregados.
gr_auditor	Auditoria interna/externa	Lectura de auditoria, bitacoras y evidencia; sin modificacion.
gr_dba_admin	Administracion tecnica	Mantenimiento, backups, roles y cambios controlados.

3. Matriz De Permisos PostgreSQL

Esquema / Dominio	App writer	Operaciones	Cumplimiento	Riesgo	Finanzas	Analitica	Auditor	DBA
reference	R	R	R	R	R	R	R	RW
customer	RW limitado	R enmascarado	R	R enmascarado	R limitado	R enmascarado	R	RW
fx	RW cotizacion	R	R	R	R	R	R	RW
remittance	RW operativo	R	R	R	R	R enmascarado	R	RW
compliance	W controlado	No directo	RW	R	No directo	R enmascarado	R	RW
settlement	R limitado	R	R limitado	No directo	RW	R agregado	R	RW
integration	RW outbox/idempotencia	R limitado	R limitado	R limitado	R limitado	R limitado	R	RW
security_audit	No directo	No directo	R limitado	No directo	No directo	No directo	R	RW

Leyenda:

R = lectura.
RW = lectura y escritura.
Limitado = solo objetos necesarios.
Enmascarado = vistas sin datos sensibles completos.
No directo = sin privilegios directos.

4. Datos Sensibles Y Restricciones

Dato	Tabla / coleccion	Control
Numero de documento	<code>customer.party_document.document_number</code>	No exponer en vistas analiticas; cifrar o tokenizar en produccion.
Cuenta de pago	<code>customer.beneficiary_payout_method.account_identifier_encrypted</code>	Cifrado; solo mostrar <code>account_identifier_masked</code> .
Travel Rule	<code>compliance.travel_rule_record</code>	Acceso solo a cumplimiento, auditoria y DBA.
AML/KYC	<code>compliance.aml_*</code> , <code>compliance.kyc_profile</code>	Acceso por cumplimiento y auditoria.
Montos de remesa	<code>remittance.remittance_financial_detail</code>	Confidencial; lectura por operacion, finanzas y analitica controlada.
Eventos Outbox	<code>integration.outbox_event</code>	Acceso tecnico, sin exposicion a usuarios finales.

5. Roles MongoDB Propuestos

Rol	Base	Permisos	Uso
<code>gr_mongo_analytics_read</code>	<code>globalremit_analytics</code>	<code>find</code> sobre colecciones analiticas	Reporteria y dashboards.
<code>gr_mongo_pipeline_rw</code>	<code>globalremit_analytics</code>	<code>find</code> , <code>insert</code> , <code>update</code> , <code>remove</code> controlado	Carga desde pipeline/generador.
<code>gr_mongo_compliance_read</code>	<code>globalremit_analytics</code>	<code>find</code> en <code>fraud_signals</code> y <code>compliance_case_snapshots</code>	Cumplimiento y riesgo.
<code>gr_mongo_audit_read</code>	<code>globalremit_analytics</code>	<code>find</code> en colecciones de evidencia	Auditoria.

6. Segregacion De Funciones

- El rol de aplicacion no administra roles ni esquemas.
- Los analistas consultan vistas o colecciones derivadas, no tablas sensibles completas.
- Cumplimiento puede revisar AML/KYC, pero no modificar liquidaciones.
- Finanzas puede operar liquidaciones, pero no modificar datos KYC/AML.
- Auditoria observa y verifica, no cambia datos operativos.
- DBA administra infraestructura, pero sus acciones deben quedar auditadas.

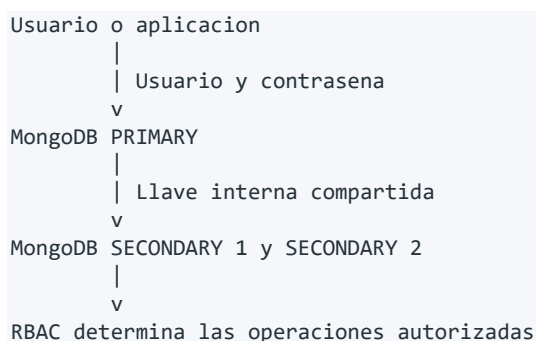
Etapa 4 - Autenticacion Y RBAC En MongoDB Replica Set

1. Objetivo

Habilitar autenticacion real en el Replica Set de MongoDB de GlobalRemit y aplicar control de acceso basado en roles (RBAC). Esta guia incluye comandos, resultados esperados y puntos para incorporar evidencia.

TLS no se activa en esta guia. El cifrado de red se documenta en [10_cifrado_tls_reposo.md](#).

2. Concepto De Seguridad



La opcion `--auth` obliga a usuarios y aplicaciones a identificarse. El `mongo-keyfile` es una credencial compartida exclusivamente por los servidores: permite que `mongo1`, `mongo2` y `mongo3` se reconozcan como miembros validos. La llave no es una contrasena para personas y nunca debe mostrarse en capturas, documentos ni repositorios.

Usuario	Rol	Acceso esperado
gr_mongo_admin	root	Administracion completa del laboratorio.
gr_mongo_analyst	gr_mongo_analytics_read	Lectura analitica.
gr_mongo_pipeline	gr_mongo_pipeline_rw	Lectura y escritura del pipeline.
gr_mongo_compliance	gr_mongo_compliance_read	Lectura limitada de cumplimiento.

3. Precauciones

- No ejecutar `docker compose down -v`.
- No borrar los volúmenes de MongoDB.
- No publicar el archivo `mongo-keyfile`.
- No activar `--auth` antes de crear un administrador.
- No usar las credenciales academicas en produccion.

Todos los comandos se ejecutan desde PowerShell:

```
cd D:\umg\UMG\Tarea2
```

4. Evidencia Inicial

Comprobar los contenedores y el estado inicial del Replica Set:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker exec globalremit-mongo1 mongosh --quiet --eval "rs.status().members.map(m =>
({name:m.name,state:m.stateStr,health:m.health}))"
```

Resultado esperado: un **PRIMARY**, dos **SECONDARY** y **health: 1**.

Evidencia MONGO-01: Replica Set saludable antes del cambio.

5. Crear El Administrador Inicial

Debe crearse antes de activar **--auth**, para no dejar el cluster protegido pero sin una cuenta capaz de administrarlo.

```
docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-
mongo2:27017,globalremit-mongo3:27017/admin?replicaSet=rsGlobalRemit" --eval
"db.createUser({user:'gr_mongo_admin',pwd:'GlobalRemitMongoAdmin2026!',roles:[{role:'root',db:'a
dmin'}]}")"
```

Resultado esperado: **ok: 1**.

```
docker exec globalremit-mongo1 mongosh --quiet --eval
"db.getSiblingDB('admin').getUser('gr_mongo_admin')"
```

Evidencia MONGO-02: usuario administrador creado. Ocultar la contraseña en la entrega. Si ya existe, no repetir createUser.

6. Generar La Llave Interna

```
New-Item -ItemType Directory -Force ".\Migración\secrets"

$mongoKeyBytes = New-Object byte[] 756
$mongoRng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$mongoRng.GetBytes($mongoKeyBytes)
$mongoKeyPath = Join-Path (Get-Location) "Migración\secrets\mongo-keyfile"
[IO.File]::WriteAllText(
    $mongoKeyPath,
    [Convert]::ToBase64String($mongoKeyBytes)
)
$mongoRng.Dispose()
```

Verificar sin mostrar el contenido:

```
Get-Item ".\Migración\secrets\mongo-keyfile" |
Select-Object FullName, Length, LastWriteTime
```

El tamaño esperado es aproximadamente 1008 bytes. Se debe agregar a **.gitignore**:

```
Migración/secrets/
```

Evidencia MONGO-03: propiedades del archivo, sin mostrar la llave.

7. Configurar Los Tres Nodos

Editar `Migración/docker-compose.yml`. En cada servicio MongoDB se debe copiar la llave dentro del contenedor, restringir sus permisos y activar `--auth` y `--keyFile`.

Ejemplo para `mongo1`:

```
mongo1:
  image: mongo:8
  container_name: globalremit-mongo1
  entrypoint: ["bash", "-lc"]
  command:
    - |
      cp /run/secrets/mongo-keyfile /tmp/mongo-keyfile
      chown mongod:mongod /tmp/mongo-keyfile
      chmod 400 /tmp/mongo-keyfile
      exec /usr/local/bin/docker-entrypoint.sh mongod \
        --replSet rsGlobalRemit \
        --bind_ip_all \
        --port 27017 \
        --auth \
        --keyFile /tmp/mongo-keyfile
  ports:
    - "37017:27017"
  volumes:
    - globalremit_mongo1_data:/data/db
    - ../workspace:ro
    - ./secrets/mongo-keyfile:/run/secrets/mongo-keyfile:ro
  networks:
    - globalremit-net
```

Aplicar lo mismo conservando los valores propios de cada nodo:

Servicio	Puerto host	Volumen
mongo1	37017	globalremit_mongo1_data
mongo2	37018	globalremit_mongo2_data
mongo3	37019	globalremit_mongo3_data

Los tres deben montar exactamente la misma llave:

```
- ./secrets/mongo-keyfile:/run/secrets/mongo-keyfile:ro
```

Validar la sintaxis:

```
docker compose -f .\Migración\docker-compose.yml config --quiet
```

8. Recrear Sin Borrar Datos

```
docker compose -f .\Migración\docker-compose.yml up -d --force-recreate mongo1 mongo2 mongo3
```

No se vuelve a ejecutar `mongo-init`: el Replica Set permanece configurado en los volúmenes.

```
docker ps --filter "name=globalremit-mongo" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker logs --tail 30 globalremit-mongo1
docker logs --tail 30 globalremit-mongo2
docker logs --tail 30 globalremit-mongo3
```

Evidencia MONGO-04: los tres contenedores activos.

9. Demostrar El Bloqueo Anonimo

```
docker exec globalremit-mongo1 mongosh --quiet --eval
"db.getSiblingDB('globalremit_analytics').remittance_events.findOne()"
```

Resultado esperado:

```
MongoServerError: Command find requires authentication
```

El error demuestra que el control funciona: un usuario anonimo ya no puede leer datos.

Evidencia MONGO-05: acceso anonimo rechazado.

10. Acceso Administrativo

```
docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-
mongo2:27017,globalremit-mongo3:27017/admin?replicaSet=rsGlobalRemit" -u gr_mongo_admin -p --
authenticationDatabase admin
```

La contraseña se solicita de forma interactiva. Dentro de **mongosh**:

```
db.runCommand({ connectionStatus: 1 })
rs.status().members.map(m => ({
  servidor: m.name,
  estado: m.stateStr,
  salud: m.health
})))
```

Evidencia MONGO-06: administrador autenticado y Replica Set saludable.

11. Crear Roles Funcionales

Ejecutar una sola vez **10_mongodb_rbac.js**:

```
docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-
mongo2:27017,globalremit-mongo3:27017/admin?replicaSet=rsGlobalRemit" -u gr_mongo_admin -p --
authenticationDatabase admin /workspace/10_mongodb_rbac.js
```

Consultar los usuarios y roles:

```
docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-
mongo2:27017,globalremit-mongo3:27017/admin?replicaSet=rsGlobalRemit" -u gr_mongo_admin -p --
authenticationDatabase admin --eval "db.getUsers().users.map(u =>
({user:u.user,roles:u.roles}))"
```

Evidencia MONGO-07: usuarios y roles creados.

12. Probar Menor Privilegio

Ingresar como analista:


```
docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-mongo2:27017,globalremit-mongo3:27017/globalremit_analytics?replicaSet=rsGlobalRemit" -u gr_mongo_analyst -p --authenticationDatabase admin
```

Lectura permitida:

```
db.remittance_events.findOne()
```

Escritura que debe ser rechazada:

```
db.remittance_events.insertOne({
  security_test: true,
  created_at: new Date()
})
```

Resultado esperado: **MongoServerError: not authorized.**

Evidencia MONGO-08: lectura permitida y escritura rechazada para el mismo usuario.

13. Failover Con Autenticacion

Ingresa como administrador e identifica el primario:

```
rs.status().members
  .filter(m => m.stateStr === "PRIMARY")
  .map(m => ({ name: m.name, state: m.stateStr })))
```

Desde otra ventana, detener el contenedor identificado. Ejemplo:

```
docker stop globalremit-mongo1
```

Esperar la eleccion y consultar nuevamente:

```
rs.status().members.map(m => ({
  servidor: m.name,
  estado: m.stateStr,
  salud: m.health
})))
```

Un secundario debe convertirse en **PRIMARY**. Recuperar el nodo:

```
docker start globalremit-mongo1
```

El nodo recuperado normalmente vuelve como **SECONDARY** y se sincroniza.

Evidencia MONGO-09: estado anterior, nuevo PRIMARY y nodo reincorporado.

14. Problemas Frecuentes

Problema	Revision
requires authentication	Usar -u, -p y --authenticationDatabase admin.
Authentication failed	Verificar usuario, contraseña y authSource=admin.
Error del keyfile	Confirmar chown mongodb:mongodb y chmod 400.
Nodo fuera del Replica Set	Confirmar la misma llave y revisar docker logs.
Usuario o rol existente	No repetir scripts createUser o createRole.

15. Matriz De Evidencias

Codigo	Evidencia	Resultado esperado
MONGO-01	Replica Set inicial	Un PRIMARY y dos SECONDARY.
MONGO-02	Administrador	gr_mongo_admin creado.
MONGO-03	Llave interna	Archivo existente sin revelar contenido.
MONGO-04	Contenedores	Tres nodos activos.
MONGO-05	Acceso anonimo	Consulta rechazada.
MONGO-06	Acceso administrativo	Autenticacion correcta.
MONGO-07	Usuarios RBAC	Roles funcionales creados.
MONGO-08	Menor privilegio	Lectura permitida y escritura rechazada.
MONGO-09	Failover	Nuevo PRIMARY y nodo recuperado.

16. Criterio De Cierre

Los tres nodos usan la misma llave interna.
El acceso anonimo es rechazado.
El administrador puede gestionar el cluster.
Los usuarios funcionales tienen permisos limitados.
El analista puede leer, pero no escribir.
El Replica Set conserva el failover automatico.
Las evidencias MONGO-01 a MONGO-09 fueron registradas.

Etapa 4 - Cifrado En Transito Y En Reposo

1. Objetivo

Definir controles para proteger la confidencialidad de los datos de GlobalRemit mientras viajan por la red y mientras permanecen almacenados.

2. Cifrado En Transito

2.1 PostgreSQL Patroni

En produccion, las conexiones hacia PostgreSQL deben exigir TLS desde:

```
Django API Gateway -> HAProxy -> PostgreSQL Leader  
pgAdmin / psql -> HAProxy -> PostgreSQL Leader
```

Configuracion recomendada:

Componente	Control
PostgreSQL	<code>ssl = on</code> , certificado de servidor, llave privada protegida.
HAProxy	Modo TCP con verificacion de salud; opcionalmente terminacion TLS si se define una PKI interna.
Clientes	<code>sslmode=require</code> o <code>sslmode=verify-full</code> en produccion.
Patroni	Replicacion protegida dentro de red privada; TLS recomendado si cruza hosts fisicos o redes no confiables.

Ejemplo de cadena de conexion segura:

```
postgresql://usuario:password@host:55434/globalremit?sslmode=require
```

2.2 MongoDB Replica Set

MongoDB debe operar con TLS habilitado cuando los nodos se distribuyen entre servidores:

```
mongod --tlsMode requireTLS --tlsCertificateKeyFile /certs/mongo.pem --tlsCAFile /certs/ca.pem
```

URI recomendada:

```
mongodb://mongo1:27017,mongo2:27017,mongo3:27017/globalremit_analytics?replicaSet=rsGlobalRemit&tls=true
```

3. Cifrado En Reposo

3.1 Volúmenes Y Sistema Operativo

El cifrado en reposo se recomienda a nivel de disco/volumen:

Plataforma	Control
Windows local	BitLocker para discos donde residan volúmenes Docker.
Linux servidor	LUKS, cifrado de volumen cloud o almacenamiento cifrado.
Docker volumes	Residir sobre discos cifrados; restringir acceso al host.
Backups	Cifrado antes de moverlos fuera del host.

3.2 Datos Sensibles A Nivel Aplicacion

El modelo ya separa campos cifrados y enmascarados:

Campo	Control
<code>customer.beneficiary_payout_method.account_identifier_encrypted</code>	Guardar valor cifrado; mostrar solo <code>account_identifier_masked</code> .
<code>compliance.travel_rule_record.originator_document_number_encrypted</code>	Guardar hash/cifrado del documento.
<code>compliance.travel_rule_record.beneficiary_document_number_encrypted</code>	Guardar hash/cifrado del documento.

En ambiente académico se usa hash SHA-256 en el generador. En producción se recomienda cifrado autenticado con rotación de llaves mediante KMS, Vault o servicio equivalente.

4. Gestion De Secretos

No se deben dejar contraseñas reales en scripts o repositorios. Para el laboratorio se documentan credenciales académicas, pero el diseño productivo debe usar:

```
Docker secrets
variables de entorno protegidas
HashiCorp Vault / Azure Key Vault / AWS Secrets Manager
rotación periódica de claves
cuentas separadas por servicio
```

5. Evidencia Esperada

Para evidenciar cifrado se recomienda documentar:

1. Captura de configuración TLS o cadena con `sslmode=require`.
2. Captura de volumen o disco cifrado.
3. Consulta que muestre datos enmascarados en vez de cuentas/documentos completos.
4. Evidencia de que las contraseñas no se exponen en la aplicación final.

Etapa 4 - Politica De Enmascaramiento Y Anonimizacion

1. Objetivo

Reducir la exposicion de datos personales y financieros en ambientes no productivos, reporteria y analitica, manteniendo utilidad operativa sin revelar informacion sensible completa.

2. Datos A Proteger

Categoria	Ejemplos	Tecnica
Identidad personal	nombres, documentos, nacionalidad	Enmascaramiento parcial o pseudonimizacion.
Contacto	telefono, correo, direccion	Ocultamiento parcial.
Cuentas de pago	cuentas bancarias, billeteras	Mostrar solo ultimos cuatro caracteres.
Travel Rule	documentos y referencias de originador/beneficiario	Acceso restringido y hashes.
Datos AML/KYC	alertas, decisiones, severidad	Acceso solo por rol.
Montos	remesas y pagos	Agregacion o rangos para analitica no sensible.

3. Reglas De Enmascaramiento

Campo	Regla
legal_name	Mostrar inicial y etiqueta anonima.
document_number	Mostrar prefijo fijo y hash parcial.
contact_value EMAIL	Mantener dominio, ocultar usuario.
contact_value PHONE/MOBILE	Mostrar ultimos 4 digitos.
account_identifier_masked	Mantener solo mascara ya existente.
originator_document_number_encrypted	No exponer valor completo.
beneficiary_document_number_encrypted	No exponer valor completo.
send_amount, payout_amount	Mostrar solo en roles autorizados; usar agregados para analitica general.

4. Vistas Seguras

El archivo `10_enmascaramiento_anonimizacion.sql` crea vistas en el esquema `security_masked`:

```
security_masked.customer_profile_masked  
security_masked.remittance_order_masked  
security_masked.compliance_case_masked
```

Estas vistas sirven para analistas, reporteria academica y ambientes donde no debe mostrarse informacion sensible directa.

5. Anonimizacion Para Desarrollo

Para ambientes de desarrollo se recomienda:

1. Reemplazar nombres reales por identificadores sinteticos.
2. Reemplazar documentos por hashes irreversibles.
3. Reemplazar correos por dominios de prueba.
4. Mantener distribuciones de montos y fechas, pero sin datos personales.
5. No copiar datos AML/KYC reales si no existe autorizacion explicita.

6. Gobierno Del Dato

- El area de cumplimiento es propietaria de datos AML, KYC y Travel Rule.
- Operaciones es propietaria del ciclo de remesa.
- Finanzas es propietaria de liquidaciones y movimientos.
- Analitica consume productos derivados y enmascarados.
- Auditoria verifica acceso y cambios, pero no modifica datos.

Etapa 4 - Configuración De Auditoría

1. Objetivo

Registrar operaciones críticas sobre la capa de datos para soportar trazabilidad, investigación de incidentes, cumplimiento y revisión de controles.

2. Eventos A Auditar

Evento	Motor	Justificación
Cambios DDL	PostgreSQL y MongoDB	Detectar alteraciones de estructura.
Cambios en roles/permisos	PostgreSQL y MongoDB	Controlar privilegios.
Inserciones y cambios en remesas	PostgreSQL	Trazabilidad operativa.
Cambios en AML/KYC/Travel Rule	PostgreSQL	Cumplimiento financiero.
Cambios en liquidaciones	PostgreSQL	Control financiero.
Fallos de autenticación	Ambos	Detección de acceso no autorizado.
Cambios en colecciones analíticas	MongoDB	Trazabilidad de productos derivados.

3. Auditoría PostgreSQL

La auditoría académica se implementa con un esquema `security_audit`, una tabla de eventos y triggers sobre tablas sensibles. Para producción se recomienda complementar con `pgaudit` y centralizar logs.

Archivo asociado:

```
10_auditoria_postgresql.sql
```

Tablas iniciales auditadas:

```
customer.party
customer.party_document
customer.beneficiary_payout_method
remittance.remittance_order
remittance.remittance_financial_detail
remittance.remittance_financial_movement
compliance.aml_alert
compliance.risk_assessment
compliance.travel_rule_record
settlement.settlement_batch
integration.outbox_event
```

4. Auditoria MongoDB

MongoDB Community no incluye todas las capacidades de auditoria empresarial. En produccion se recomienda MongoDB Enterprise Auditing o logs centralizados. Para el proyecto academico se documentan:

```
rs.status()
db.hello()
logs de contenedor
bitacora de pruebas de failover
controles de acceso por roles
```

Comandos utiles:

```
docker logs globalremit-mongo1
docker exec globalremit-mongo2 mongosh --quiet --eval "db.hello()"
docker exec globalremit-mongo2 mongosh --quiet --eval "rs.status().members.map(m =>
({name:m.name,stateStr:m.stateStr,health:m.health}))"
```

5. Retencion

Evidencia	Retencion recomendada
Auditoria transaccional critica	1 a 5 anos segun regulacion aplicable.
Logs operativos	90 a 180 dias.
Evidencia de cambios de rol	1 ano minimo.
Bitacoras de pruebas academicas	Durante todo el ciclo del proyecto.

6. Revision

La auditoria debe revisarse:

- despues de cada despliegue;
- despues de cada failover o switchover;
- ante alertas de acceso no autorizado;
- durante auditorias internas o externas;
- antes de entregar reportes regulatorios.

Estado de avance de la etapa 4

Estado actual: los controles de PostgreSQL ya cuentan con comandos y evidencias; el endurecimiento de MongoDB fue iniciado con la creación del administrador y todavía requiere completar autenticación, RBAC, failover seguro y evidencias de cifrado.

Control	Estado	Evidencia disponible	Qué falta
Gobernanza y Data Mesh	Documentado	Dominios, propietarios, clasificación y ciclo de vida definidos.	Revisión de formato final.
RBAC PostgreSQL	Implementado	Roles creados y listado de roles capturado.	Probar el API Gateway con gr_api_gateway en lugar de postgres.
Enmascaramiento	Implementado	Vista enmascarada consultada y acceso directo rechazado.	Conservar ambas capturas como evidencia.
Auditoría PostgreSQL	Implementado	Triggers instalados y consulta de audit_event capturada.	Relacionar una inserción de remesa con su evento auditado.
Administrador MongoDB	En curso	gr_mongo_admin fue creado en la base admin.	Agregar captura del resultado y evitar mostrar la contraseña.
Keyfile MongoDB	Pendiente de evidencia	Comandos preparados.	Generar la llave, protegerla y capturar solamente sus propiedades.
Auth y RBAC MongoDB	Pendiente	Guía y script disponibles.	Activar --auth/--keyFile, recrear nodos y probar accesos permitidos/denegados.
Failover MongoDB autenticado	Pendiente	Escenario documentado.	Detener el PRIMARY, evidenciar elección y reincorporación.
TLS en tránsito	Documentado	Diseño para PostgreSQL y MongoDB definido.	Implementar TLS o declarar explícitamente que queda como diseño productivo.
Cifrado en reposo y secretos	Documentado	Controles BitLocker/volumenes/Vault descritos.	Adjuntar evidencia de BitLocker y eliminar secretos visibles del documento final.

Próximos pasos recomendados

Completar la generación y protección del mongo-keyfile.

Activar --auth y --keyFile en los tres nodos MongoDB.

Ejecutar las evidencias MONGO-05 a MONGO-09.

Decidir si TLS se implementará en el laboratorio o quedará documentado como arquitectura productiva.

Actualizar esta tabla de avance antes de entregar.

Anexo A - Comandos utilizados y evidencias

Este anexo conserva los comandos ejecutados y las capturas disponibles. Las credenciales visibles deben ocultarse antes de la entrega final.

Comandos utilizados

```
cd "D:\umg\UMG\Tarea2"
```

```
docker compose -f .\Migración\docker-compose.yml up -d
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -f .\10_rbac_postgresql.sql
```

```
PS D:\umg\UMG\Tarea2> psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -c "SELECT rolname FROM pg_roles WHERE rolname LIKE 'gr_%'
ORDER BY rolname;"
Contraseña:

      rolname
-----
gr_analyst_user
gr_api_gateway
gr_app_writer
gr_auditor
gr_auditor_user
gr_compliance_officer
gr_compliance_user
gr_data_analyst
gr_dba_admin
gr_finance_settlement
gr_finance_user
gr_ops_readonly
gr_ops_user
gr_risk_analyst
gr_risk_user
(15 filas)
```

```
psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -c "SELECT rolname FROM pg_roles WHERE
rolname LIKE 'gr_%' ORDER BY rolname;"
```

```
psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -f .\10_enmascaramiento_anonimizacion.sql
psql -h 127.0.0.1 -p 55434 -U gr_analyst_user -d globalremit -W -c "SELECT * FROM
security_masked.customer_profile_masked LIMIT 5;"
```

```
PS D:\umg\UMG\Tarea2> psql -h 127.0.0.1 -p 55434 -U gr_analyst_user -d globalremit -W -c "SELECT * FROM security_masked.customer_profile
masked LIMIT 5;"
Contraseña:

customer_id | customer_code | legal_name_masked | party_type | party_status | customer_status |      created_at
-----
1 | CUST-000001 | L*** 1 | INDIVIDUAL | ACTIVE | ACTIVE | 2026-07-10 15:32:26.318886+00
2 | CUST-000002 | R*** 2 | INDIVIDUAL | ACTIVE | ACTIVE | 2026-07-10 15:32:26.318886+00
3 | CUST-000003 | F*** 3 | INDIVIDUAL | ACTIVE | ACTIVE | 2026-07-10 15:32:26.318886+00
4 | CUST-000004 | V*** 4 | INDIVIDUAL | ACTIVE | ACTIVE | 2026-07-10 15:32:26.318886+00
5 | CUST-000005 | L*** 5 | INDIVIDUAL | ACTIVE | ACTIVE | 2026-07-10 15:32:26.318886+00
(5 filas)
```

```
PS D:\umg\UMG\Tarea2> psql -h 127.0.0.1 -p 55434 -U gr_analyst_user -d globalremit -W -c "SELECT * FROM customer.party LIMIT 1;"
Contraseña:
ERROR: permission denied for schema customer
LÍNEA 1: SELECT * FROM customer.party LIMIT 1;
PS D:\umg\UMG\Tarea2>
```

```
psql -h 127.0.0.1 -p 55434 -U gr_analyst_user -d globalremit -W -c "SELECT * FROM customer.party LIMIT 1;"
```

```
psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -f .\10_auditoria_postgresql.sql
psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -c "SELECT
event_time,database_user,schema_name,table_name,operation,row_pk FROM security_audit.audit_event ORDER
BY audit_event_id DESC LIMIT 15;"
```

```
PS D:\umg\UMG\Tarea2> psql -h 127.0.0.1 -p 55434 -U postgres -d globalremit -W -c "SELECT event_time,database_user,schema_name,table_name,operation,row_pk FROM secur
ity_audit.audit_event ORDER BY audit_event_id DESC LIMIT 15;"
Contraseña:
```

event_time	database_user	schema_name	table_name	operation	row_pk
2026-07-23 18:32:08.019445+00	postgres	integration	outbox_event	INSERT	c2c66fcd-59c7-4aa9-be4a-1212565ef942
2026-07-23 18:32:08.019445+00	postgres	compliance	risk_assessment	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_financial_movement	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_financial_movement	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_financial_movement	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_financial_movement	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_financial_movement	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	compliance	travel_rule_record	INSERT	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555
2026-07-23 18:32:08.019445+00	postgres	remittance	remittance_order	UPDATE	1555

(15 filas)

Autenticación MongoDB

Comandos utilizados

```
cd "D:\umg\UMG\Tarea2"

docker exec -it globalremit-mongo1 mongosh "mongodb://globalremit-mongo1:27017,globalremit-
mongo2:27017,globalremit-mongo3:27017/admin?replicaSet=rsGlobalRemit" --eval
"db.createUser({user:'gr_mongo_admin',pwd:'<CONTRASENA_ADMIN_MONGODB>',roles:[{role:'root',db:'admin'}
]})"
```

```
New-Item -ItemType Directory -Force ".\Migración\secrets"
$mongoKeyBytes = New-Object byte[] 756
$mongoRng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$mongoRng.GetBytes($mongoKeyBytes)
$mongoKeyPath = Join-Path (Get-Location) "Migración\secrets\mongo-keyfile"
[IO.File]::WriteAllText(
    $mongoKeyPath,
    [Convert]::ToBase64String($mongoKeyBytes)
)
$mongoRng.Dispose()
```